Homework #6

[ECE30021] Operating Systems

Mission

- Solve problem 1, 2, and 3
  - Solve the problems on Ubuntu Linux (VirtualBox or UTM) using vim and gcc.
  - Problem 1: discussion within study group and using AI are allowed
  - Problems 2 and 3: for individual students (discussion and use of Internet/AI are prohibited)

- Submission
  - Submit a .tgz file containing hw6_1.c, hw6_2.c and hw6_3.c on LMS.
    "tar cvfz hw6_<*student_id*>.tgz hw6_1.c hw6_2.c hw6_3.c"
    (Do not copy&past but type the above command manually!)
  - After compression, please check the .tgz file by decompressing in an empty directory.
    "tar xvfz hw6_<*student_id*>.tgz"

- Due date: PM 11:00:00, June 14th

Honor Code Guidelines

- “과제”
  - 과제는 교과과정의 내용을 소화하여 실질적인 활용 능력을 갖추기 위한 교육활동이다. 학생은 모든 과제를 정직하고 성실하게 수행함으로써 과제에 의도된 지식과 기술을 얻기 위해 최선을 다해야 한다.
  - 제출된 과제물은 성적 평가에 반영되므로 공식적으로 허용되지 않은 자료나 도움을 획득, 활용, 요구, 제공하는 것을 포함하여 평가의 공정성에 영향을 미치는 모든 형태의 부정행위는 단호히 거부해야 한다.
  - 수업 내용, 공지된 지식 및 정보, 또는 과제의 요구를 이해하기 위하여 동료의 도움을 받는 것은 부정행위에 포함되지 않는다. 그러나, 과제를 해결하기 위한 모든 과정은 반드시 스스로의 힘으로 수행해야 한다.
  - 담당교수가 명시적으로 허락한 경우를 제외하고 다른 사람, 또는 AI가 작성하였거나 인터넷 등에서 획득한 과제물, 또는 프로그램 코드의 일부, 또는 전체를 이용하는 것은 부정행위에 해당한다.
  - 자신의 과제물을 타인에게 보여주거나 빌려주는 것은 공정한 평가를 방해하고, 해당 학생의 학업 성취를 저해하는 부정행위에 해당한다.
  - 팀 과제가 아닌 경우 두 명 이상이 함께 과제를 수행하여 이를 개별적으로 제출하는 것은 부정행위에 해당한다.
  - 스스로 많은 노력을 한 후에도 버그나 문제점을 파악하지 못하여 동료의 도움을 받는 경우도 단순한 문법적 오류에 그쳐야 한다. 과제가 요구하는 design, logic, algorithm의 작성에 있어서 담당교수, TA, tutor 이외에 다른 사람의 도움을 받는 것은 부정행위에 해당한다.
  - 서로 다른 학생이 제출한 제출물간 유사도가 통상적으로 발생할 수 있는 정도를 크게 넘어서는 경우, 또는 자신이 제출한 과제물에 대하여 구체적인 설명을 하지 못하는 경우에는 부정행위로 의심받거나 판정될 수 있다.

Problem 0

- Learn how to use the bit-wise operators by watching the lecture video.
  - https://hducc.handong.edu/em/683d89ee1f646

Problem 1

- Mission: peer review of HW#5 problems as a group
  - Review the solutions of other members and share comments to each other.
  - Each member update her/his solutions individually applying the comments.
    - Name the updated solution to Problem 3 as hw6_1_<student_ID>.c
    - Mark the modified parts by comments as the next page.
    - If you believe your previous solution was perfect, submit it again and put a comment "My previous solution was perfect, so I didn't modify it!" in the first line.
  - In hw6_1_<student_ID>.c, write the advices you gave to your group mates as comments. (Hangul comments are okay.)

```
/* Examples of comments to other members
        Comments to other_member_1:
          - Line ##: This line is buggy. Fix in this way…
          - Line ##: This line can be rewritten more efficiently like …
          …
        Comments to other_member_2:
          - Line ##: Initialize the variable i
          - Line ##: Use increase the size of aaaa_array
*/
```

Problem 1

- Example of updated solution.

```
int main()
{
	int i = 0;

	// previous code
	/*
	<old code>

	*/

	// updated code
	<new code>


	return 0;
}
```

Problem 2

- Implement the address translation algorithm of paging
  - Read a page table from a file specified in the command-line argument
    - Usage: ./hw6_2 <page_table>
  - Implement the following functions
    - int ReadTable(const char *filename, int verbose);
      - Read a page table from the specified file
    - void FreeTable();
      - Deallocate page table
    - int l2p(int la, int verbose);
      - Translate logical address into physical address
  - Make the output messages as similar to the examples as possible

Problem 2

- Global variables for page table
  - int m = 0; // see pp. 27 of chap. 9 lecture note
  - int n = 0; // see pp. 27 of chap. 9 lecture note
  - int *ptbr = NULL; // see pp. 35 of chap. 9 lecture note
  - int ptlr = 0; // see pp. 39 of chap. 9 lecture note
  - int mask = 0; // a binary mask to extract offset from logical address
    // e.g., offset = <logical_address> & mask;
    // if n == 2, mask should be 3_{10} (11_2)
    // if n == 3, mask should be 7_{10} (111_2)

- The format of the page table file

```
4                   // m
2                   // n
4                   // PTLR

5                   // page table entries
6
1
2
```

0	5
1	6
2	1
3	2

Problem 2

- int ReadTable(const char *filename, int verbose);
  - Read a page table.
    - Dynamically allocate memory for the page table
    - On success, return TRUE
    - On failure, display an error message and return FALSE
    - If verbose is TRUE, display the page table

```
page table
        m = 4, n = 2
        page size = 4
        maximum number of pages = 4
        ptlr = 4
        offset mask = 0x3   // display as a hexadecimal number
        ptbr[0] = 5
        ptbr[1] = 6
        ptbr[2] = 1
        ptbr[3] = 2
```

Problem 2

- void FreeTable();
  - Deallocate the page table and reset ptbr to NULL
  - Reset m, n, ptlr, and mask to zero
- int l2p(int la, int verbose);
  - Return the physical address correspond to la
  - Implement the algorithm illustrated in pp. 29 of chap. 9 lecture note.
    - Extract the page number from the logical address using the right shift operator (>>)
    - Extract the offset from the logical address using the bit-wise and operator (&)
    - Lookup the page table to convert the page number into the corresponding frame number
    - Generate the physical address from the frame number and offset using the left shift operator (<<) and the bit-wise or operator (|)
  - If verbose is TRUE, display the following values
    - Logical address, page number, frame number, offset, (frame_number << n), and physical address

Problem 2

- The main() function

```
int main(int argc, char *argv[])
{
    if(argc <= 1){
        printf("Usage: %s <page_table>₩n", argv[0]);
        return 0;
    }

    int ret = ReadTable(argv[1], TRUE);
    if(ret == FALSE)
        exit(-1);

    int step = 1;
    int max_la = ptlr * (1 << n);
    if(max_la > 20)
        step = max_la / 20;

    printf("max_la = 0x%x (%d)₩n", max_la, max_la);

    for(int la = 0; la <= max_la; la += step){
        int pa = l2p(la, FALSE);
        printf("₩t0x%x (%d)--> 0x%x (%d)₩n", la, la, pa, pa);
    }

    FreeTable();

    printf("Bye!₩n");

    return 0;
}
```

```
page table
        m = 4, n = 2
        page size = 4
        maximum number of pages = 4
        ptlr = 4
        offset mask = 0x3
        ptbr[0] = 5
        ptbr[1] = 6
        ptbr[2] = 1
        ptbr[3] = 2
max_la = 0x10 (16)
        0x0 (0)--> 0x14 (20)
        0x1 (1)--> 0x15 (21)
        0x2 (2)--> 0x16 (22)
        0x3 (3)--> 0x17 (23)
        0x4 (4)--> 0x18 (24)
        0x5 (5)--> 0x19 (25)
        0x6 (6)--> 0x1a (26)
        0x7 (7)--> 0x1b (27)
        0x8 (8)--> 0x4 (4)
        0x9 (9)--> 0x5 (5)
        0xa (10)--> 0x6 (6)
        0xb (11)--> 0x7 (7)
        0xc (12)--> 0x8 (8)
        0xd (13)--> 0x9 (9)
        0xe (14)--> 0xa (10)
        0xf (15)--> 0xb (11)
TRAP: Page number 4 is greater than ptlr 4.
        0x10 (16)--> 0xffffffff (-1)
Bye!
```

Problem 3

- Implement the address translation algorithm of segmentation
  - Read a segment table from a file specified in the command-line argument
    - Usage: ./hw6_3 <page_table>
  - Implement the following functions
    - int ReadTable(const char *filename, int verbose);
      - Read a segement table from the specified file
    - void FreeTable();
      - Deallocate page table
    - int l2p(int sn, int offset, int verbose);
      - Translate logical address into physical address
  - Make the output messages as similar to the examples as possible

Problem 3

- Global variables for page table
  - typedef struct { // the entry of segment table
  - int limit, base;
  - } Descriptor;
  - Descriptor *stbr = NULL; // see pp. 51 of chap. 9 lecture note
  - int stlr = 0; // see pp. 51 of chap. 9 lecture note
- The format of the page table file

```
5            // STLR

1000         // the limit of each segment
400
400
1100
1000

1400         // the base address of each segment
6300
4300
3200
4700
```

	limit	base
0	1000	1400
1	400	6300
2	400	4300
3	1100	3200
4	1000	4700

Problem 3

- int ReadTable(const char *filename, int verbose);
  - Read a segment table.
    - Dynamically allocate memory for the segment table
    - On success, return TRUE
    - On failure, display an error message and return FALSE
    - If verbose is TRUE, display the segment table

```
Segment Table
        stlr = 5
        limit = 1000, base = 1400
        limit = 400, base = 6300
        limit = 400, base = 4300
        limit = 1100, base = 3200
        limit = 1000, base = 4700
```

Problem 3

- void FreeTable();
  - Deallocate the segment table and reset stbr to NULL
  - Reset stlr to zero
- int l2p(int sn, int offset, int verbose);
  - Return the physical address correspond to the logical addr. (sn, offset)
  - Implement the algorithm illustrated in pp. 50 of chap. 9 lecture note.
    - On failure, return an appropriate error message and return -1
      - e.g., sn >= stlr or offset >= limit
  - If verbose is TRUE, display the following values
    - Segment number (sn), the limit and base address of the sn^{th} segment
    - The logical address (sn, off) and the physical address

Problem 3

- The main() function

```
int main(int argc, char *argv[])
{
    if(argc == 1){
        printf("Usage: %s <segment_table>₩n", argv[0]);
        return 0;
    }

    int ret = ReadTable(argv[1], TRUE);
    if(ret == FALSE)
        exit(-1);

    for(int sn = 0; sn <= stlr; sn++){
        Descriptor *d = &stbr[sn];
        int offset[] = { 0, d->limit/2, d->limit - 1, d->limit };

        for(int j = 0; j < 4; j++)
            printf("₩t(%d, %d) -> %d₩n", sn, offset[j], l2p(sn, offset[j], FALSE));

        putchar('₩n');
    }

    FreeTable();

    return 0;
}
```

```
Segment Table
        stlr = 5
        limit = 1000, base = 1400
        limit = 400, base = 6300
        limit = 400, base = 4300
        limit = 1100, base = 3200
        limit = 1000, base = 4700

        (0, 0) -> 1400
        (0, 500) -> 1900
        (0, 999) -> 2399
TRAP: offset 1000 >= limit 1000
        (0, 1000) -> -1

        (1, 0) -> 6300
        (1, 200) -> 6500
        (1, 399) -> 6699
TRAP: offset 400 >= limit 400
        (1, 400) -> -1

        (2, 0) -> 4300
        (2, 200) -> 4500
        (2, 399) -> 4699
TRAP: offset 400 >= limit 400
        (2, 400) -> -1

        (3, 0) -> 3200
        (3, 550) -> 3750
        (3, 1099) -> 4299
TRAP: offset 1100 >= limit 1100
        (3, 1100) -> -1

        (4, 0) -> 4700
        (4, 500) -> 5200
        (4, 999) -> 5699
TRAP: offset 1000 >= limit 1000
        (4, 1000) -> -1

TRAP: segment no 5 >= stlr 5
        (5, 0) -> -1
TRAP: segment no 5 >= stlr 5
        (5, 520) -> -1
TRAP: segment no 5 >= stlr 5
        (5, 1040) -> -1
TRAP: segment no 5 >= stlr 5
        (5, 1041) -> -1
```